

ERP Deployment & Backup

SEPL ERP — Redis optimization layer (first production deploy). Staged: code first (fallback-safe), then Redis on.

1 · Pre-flight + backup (VPS)

```
cd /root/erp
node --version           # must be >= 18
git rev-parse --short HEAD # note this for rollback
git status               # MUST be clean (deploy.sh does a hard reset)
node server/scripts/backup-db.js # WAL-safe snapshot of erp.db + chat.db -> /root/erp-backups/
```

Use the script, not `cp` : SQLite runs in WAL mode, so a plain file copy can grab the DB mid-write and corrupt the backup. `backup-db.js` uses the native online-backup API (consistent snapshot while the server writes), backs up both DBs, and keeps the last 30.

2 · Deploy code (runs in fallback; Redis not on yet)

```
bash scripts/deploy.sh
pm2 status              # erp: online
```

Expected: the smoke test warns “Redis not reachable” at this stage — correct, Redis isn’t installed yet. The app serves the new code in fallback mode.

3 · Install Redis + configure .env

```
sudo bash scripts/setup-redis.sh
redis-cli ping           # -> PONG

nano /root/erp/.env      # APPEND the block below (do NOT overwrite)

grep -E 'JWT_SECRET|REDIS_URL' /root/erp/.env # confirm JWT_SECRET still present
```

Critical: append to `.env`, never overwrite. Overwriting drops `JWT_SECRET` and logs every user out.

4 · Restart on Redis + verify

```
pm2 restart erp --update-env
node server/scripts/redis-smoke.js # expect 4/4 PASS
redis-cli exists sepl:t:global:jobs:worker-alive # -> 1 (worker heartbeat)
pm2 save
```

.env block to append

```
# --- Redis optimization layer ---
REDIS_URL=redis://127.0.0.1:6379
REDIS_KEY_PREFIX=sepl
PRESENCE_TTL_SEC=30
ERP_DISABLE_REDIS=
ERP_DISABLE_JOB_QUEUE=
ERP_EMBED_WORKER=1
```

Routine deploy — every deploy AFTER this one

Once Redis is live, forget the staged rollout above. Redis is installed and `.env` already has the block, so a normal deploy is one script. No `setup-redis.sh`, no `.env` edits, no manual `pm2 stop / restart` — `deploy.sh` does a hard reset + `pm2 startOrReload`.

```
# 0) merge the feature branch to main first (PR on GitHub, or gh cli),
#    so origin/main has the commits BEFORE touching the VPS.

# on the VPS:
cd /root/erp
git rev-parse --short HEAD      # note current commit for rollback
git status                      # MUST be clean (deploy.sh does a hard reset)
node server/scripts/backup-db.js # WAL-safe snapshot of erp.db + chat.db

bash scripts/deploy.sh          # deploys origin/main: fetch+reset -> npm install
                                # (rebuilds client) -> pm2 startOrReload -> smoke test.
                                # It PRINTS the 4/4 result at the end — watch for it.

# deploy a non-main branch instead: bash scripts/deploy.sh <branch>

pm2 status                      # erp: online
# deploy.sh already ran the smoke test; re-run by hand ONLY to re-check:
#   node server/scripts/redis-smoke.js    # expect 4/4 PASS
```

Only re-run the first-time steps when they actually change: `sudo bash scripts/setup-redis.sh` only if reinstalling Redis; edit `.env` + `pm2 restart erp --update-env` only when an env var changes (`startOrReload` does **not** re-read `.env`). Routine code-only deploys need neither.

Redis stoppage & recovery (routine ops)

Not an incident. If Redis stops, the ERP keeps working — every path falls back to direct SQLite, so it's *correct, just slower*. The app also self-heals: it retries the connection forever and re-engages acceleration the moment Redis is back. **No `pm2 restart` is needed after a normal Redis restart.**

A · Redis stopped — just bring it back (normal case)

```
sudo systemctl status redis-server # confirm it's down / why
sudo systemctl restart redis-server # (or: start)
redis-cli ping                      # -> PONG
# app reconnects on its own; watch for "[redis] main ready":
pm2 logs erp --lines 20
node server/scripts/redis-smoke.js  # expect 4/4 PASS
redis-cli exists sepl:t:global:jobs:worker-alive # -> 1 (worker heartbeat back)
```

B · Redis flapping / won't stay up — stop the app leaning on it

```
# instant, no redeploy: force clean fallback (pre-Redis behavior)
echo 'ERP_DISABLE_REDIS=1' >> /root/erp/.env
pm2 restart erp --update-env

# once Redis is fixed, re-enable: set the line back to ERP_DISABLE_REDIS= (empty)
nano /root/erp/.env
pm2 restart erp --update-env
node server/scripts/redis-smoke.js    # confirm 4/4
```

C · Diagnose why it stopped

```
sudo systemctl status redis-server -l
sudo journalctl -u redis-server -n 50 --no-pager
redis-cli info memory | grep -E 'used_memory_human|maxmemory'
```

Most common cause on this 2-core box is memory pressure / the system OOM-killer. The durable fix is the 2 GB swap file noted in `ecosystem.config.js`. Redis is configured `noeviction` with no `maxmemory` cap, so it rejects writes under pressure (jobs/cache fail over to inline) rather than crashing.

Rollback

```
# Disable Redis, keep new code (instant, no redeploy):
echo 'ERP_DISABLE_REDIS=1' >> /root/erp/.env && pm2 restart erp --update-env

# Full code rollback:
git reset --hard <commit-from-step-1>
npm install
pm2 startOrReload ecosystem.config.js
```

Databases (`data/erp.db`, `data/chat.db`), `.env`, `node_modules/` and `client/dist/` are gitignored — the hard reset in `deploy.sh` never touches them. Backups from step 1 are the belt-and-suspenders copy.